

Parallel Computing - MCQ

Part 1: Question bank MID

1. How much did the performance of microprocessors increase, on average, per year from 1986 to 2003?
A) Less than 4%
B) 10%
C) 25%
D) More than 50%
2. What happened to single-processor performance improvement between the years 2015 and 2017?
A) It increased by almost a factor of 60.
B) It increased at less than 4% per year.
C) It increased by 50% per year.
D) It remained completely stagnant.
3. Why did manufacturers decide to start putting multiple complete processors on a single integrated circuit by 2005?
A) To decrease the physical size of desktop computers.
B) Because developing ever-faster monolithic processors became too difficult.
C) To eliminate the need for main memory.
D) Because parallel systems use less electricity overall.
4. Why is it becoming impossible to continue increasing the speed of integrated circuits by just making transistors smaller and faster?
A) Air-cooled integrated circuits have reached the limits of their ability to dissipate heat.
B) Transistors cannot be physically connected to modern motherboards.
C) The speed of light creates a hard limit on data transfer rates.
D) Monolithic processors are too expensive to manufacture.
5. What is the primary difficulty in relying on translation programs to automatically convert serial C, C++, or Java code into parallel programs?
A) Parallel languages do not support arrays.
B) The sequence of automatically translated parallel constructs may be terribly inefficient compared to devising a new parallel algorithm.
C) Translation programs can only be run on single-core systems.
D) Serial programs do not use loops.
6. Which approach to writing parallel programs partitions the various operations carried out in solving a problem among the cores?
A) Data-parallelism
B) Task-parallelism
C) Pipeline-parallelism
D) Serial-parallelism

7. Which approach to writing parallel programs partitions the arrays or structures used in solving a problem among the cores?

- A) Data-parallelism
- B) Task-parallelism
- C) Pipeline-parallelism
- D) Instruction-level parallelism

8. If a professor and her TAs grade an exam by having each person grade all 100 responses to one specific question, what type of parallelism does this represent?

- A) Data-parallelism
- B) SIMD-parallelism
- C) Task-parallelism
- D) Distributed-parallelism

9. If a professor and her TAs grade an exam by dividing 100 exams into five equal piles and each person grades all the questions in their pile, what type of parallelism is this?

- A) Data-parallelism
- B) Task-parallelism
- C) Instruction-level parallelism
- D) Pipeline-parallelism

10. What is the process of dividing work among cores so that each core takes roughly the same amount of time to finish called? A) Synchronization

- B) Caching
- C) Pipelining
- D) Load balancing

11. What mechanism is used to ensure that cores do not race ahead and start computing partial sums before a master core has finished initializing an array?

- A) Speculation
- B) Synchronization
- C) Caching
- D) Virtual Memory

12. In a tree-structured global sum with 1000 cores, approximately how many receives and adds does the master core require compared to a serial global sum?

- A) 999 receives and adds
- B) 500 receives and adds
- C) 10 receives and adds
- D) 100 receives and adds

13. Which API is primarily used for programming distributed memory MIMD systems?

- A) Pthreads
- B) OpenMP
- C) CUDA
- D) Message-Passing Interface (MPI)

14. Which API is a library used in C programs for programming shared memory MIMD systems?

- A) MPI

B) POSIX threads (Pthreads)

C) CUDA

D) OpenCL

15. Which API consists of a library and compiler modifications used for programming both shared memory MIMD and SIMD systems?

A) Pthreads

B) MPI

C) OpenMP

D) Hadoop

16. Which API is used for programming Nvidia GPUs?

A) CUDA

B) OpenMP

C) MPI

D) Pthreads

17. In which type of memory system can all cores read and write to all memory locations?

A) Distributed-memory system

B) Shared-memory system

C) Flash-memory system

D) Cache-only

18. In which type of memory system does each core have its own private memory, requiring explicit communication over a network?

A) Shared-memory system

B) Distributed-memory system

C) Uniform memory access system

D) Monolithic memory system

19. What distinguishes concurrent computing from parallel computing according to typical definitions?

A) In concurrent computing, multiple tasks can be in progress at any instant, while in parallel computing, multiple tasks cooperate closely to solve a problem.

B) Concurrent computing only runs on distributed systems.

C) Parallel computing requires a single core to switch between tasks rapidly.

D) There is absolutely no difference between the two terms.

20. Which of the following is NOT a component of the classical von Neumann architecture?

A) Main memory

B) Central processing unit (CPU)

C) Interconnection between memory and CPU

D) Distributed network switches

21. What is the control unit in a CPU responsible for?

A) Executing the actual instructions

B) Storing data permanently on disk

C) Deciding which instructions in a program should be executed

D) Cooling the processor

22. What does the program counter in a von Neumann architecture store?

- A) The address of the next instruction to be executed
- B) The total number of instructions executed so far
- C) The memory required by the program
- D) The ID of the currently running thread

23. What problem does the term "von Neumann bottleneck" describe?

- A) The CPU overheating due to transistor density
- B) The interconnect determining the maximum rate at which instructions and data can be accessed from memory
- C) The operating system freezing when switching processes
- D) The inability of a program to execute concurrently

24. What does a multitasking operating system provide?

- A) Hardware acceleration for single-threaded programs
- B) The apparent simultaneous execution of multiple programs via time slices
- C) Distributed memory management across a network
- D) Infinite virtual memory without disk swapping

25. Which of the following statements about threads and processes is true?

- A) Processes are lighter weight than threads.
- B) Threads are contained within processes and usually share the same memory.
- C) Switching between processes is much faster than switching between threads.
- D) Threads do not require their own call stacks.

26. Which two components must each thread possess independently of other threads in the same process?

- A) An executable machine language program and file descriptors
- B) A heap and security information
- C) A record of its own program counter and a call stack
- D) A virtual memory page table and an I/O interconnect

27. What is a CPU cache?

- A) A type of secondary storage mechanism
- B) A collection of memory locations that the CPU can access more quickly than main memory
- C) An interconnect linking multiple processors together
- D) A software buffer used by the operating system to print to stdout

28. What principle states that programs tend to use data and instructions physically close to recently used data and instructions?

- A) Amdahl's Law
- B) Principle of locality
- C) Foster's methodology
- D) Moore's Law

29. Accessing a nearby memory location in the near future after a recent access defines what concept?

- A) Instruction-level parallelism
- B) Spatial and temporal locality
- C) Virtual memory paging

D) Interleaved memory

30. What is the term for the **block** of data or instructions transferred into the cache during a memory access?

A) Cache miss

B) Cache line or cache block

C) TLB entry

D) Memory page

31. What happens when the CPU needs information that is **not available** in the cache?

A) A cache hit

B) A page fault

C) A cache miss

D) A translation-lookaside buffer (TLB) stall

32. In which caching scheme is data written to main memory at the exact same time it is written to the cache?

A) Write-back

B) Write-through

C) Fully associative

D) Direct mapped

33. In which caching scheme is updated data marked **"dirty"** and only written to memory when the cache line is evicted?

A) Write-through

B) Speculative execution

C) Direct mapped

D) Write-back

34. In a **direct mapped** cache, where can a fetched cache line be placed?

A) In any available location in the cache

B) In one of two specific locations

C) In a single unique location

D) Directly into a CPU register

35. In a fully associative cache, where can a **new line** be placed?

A) Only in index 0

B) At any location in the cache

C) At a unique location determined by modulo division

D) In the TLB A

36. Because C stores two-dimensional arrays in row-major order, which **loop** nesting order is generally faster for cache performance?

A) Iterating over columns in the outer loop and rows in the inner loop

B) Iterating over rows in the outer loop and columns in the inner loop

C) Accessing array elements randomly to avoid cache mapping conflicts

D) There is no performance difference between column-major and row-major access

37. What was developed so that main **memory** can function as a cache for secondary storage?

- A) Write-through caching
- B) Virtual memory
- C) The TLB
- D) Hardware multithreading

38. In virtual memory, what are the blocks of data and instructions called?

- A) Pages
- B) Cache lines
- C) Datapaths
- D) Vectors

39. What data structure is created to translate virtual page numbers to physical memory addresses?

- A) Cache hierarchy
- B) Program counter
- C) Page table
- D) Vector register

40. What is a Translation-Lookaside Buffer (TLB)?

- A) A buffer used for storing speculative execution results
- B) A special address translation cache that stores a small number of entries from the page table
- C) A section of main memory dedicated to I/O operations
- D) The interconnect between multiple GPUs

41. What is a page fault?

- A) An error caused by writing to a read-only variable
- B) A cache miss in the L1 cache
- C) An attempt to access a page that is not currently in main memory
- D) A collision on the system bus

42. What are the two main approaches to instruction-level parallelism (ILP)?

- A) Shared memory and distributed memory
- B) Pipelining and multiple issue
- C) Caching and virtual memory
- D) Single instruction and multiple data

43. Which ILP approach arranges functional units in stages, similar to a factory assembly line?

- A) Multiple issue
- B) Speculation
- C) Pipelining
- D) Hardware multithreading

44. Which ILP approach replicates functional units to simultaneously execute different instructions in a program?

- A) Dynamic routing
- B) Static assignment
- C) Multiple issue
- D) Pipelining

45. What technique does a **multiple issue** system use to make a guess about an instruction and execute it based on that guess?

A) **Speculation**

B) Pipelining

C) Caching

D) Swapping

46. How does **thread-level parallelism** (TLP) differ from ILP?

A) **TLP provides coarser-grained parallelism by simultaneously executing different threads.**

B) TLP relies entirely on a single thread to find pipelining opportunities.

C) TLP applies the exact same instruction to multiple data items.

D) TLP is only found in von Neumann architectures.

47. Which form of **hardware multithreading** attempts to avoid wasting machine time by switching between threads after each instruction?

A) Coarse-grained multithreading

B) **Fine-grained multithreading**

C) Virtual threading

D) Batch scheduling

48. Which form of hardware multithreading **only switches threads** when the current thread is stalled waiting for a time-consuming operation, like a memory load?

A) Fine-grained multithreading

B) Simultaneous multithreading (SMT)

C) **Coarse-grained multithreading**

D) Speculative multithreading

49. In **Flynn's taxonomy**, what does a standard classical von Neumann system classify as?

A) **SIMD**

B) MIMD

C) SISD

D) MISD

50. What type of system applies a **single instruction to multiple data** items simultaneously via multiple datapaths?

A) SISD

B) **SIMD**

C) MIMD

D) Distributed-memory

51. What is a key characteristic of **vector** processors?

A) They lack instruction caches entirely.

B) They use only scalar registers.

C) **They can operate on arrays or vectors of data using vector instructions.**

D) They exclusively use direct-mapped caching.

52. What is "strided memory access" in the context of **vector** processors?

A) **Accessing elements of a vector located at fixed intervals.**

B) Randomly accessing variables in main memory.

- C) Fetching cache lines backwards.
- D) Moving data exclusively via a crossbar switch.

53. What parallel processing classification best describes the programmable shader functions in modern Graphics Processing Units (GPUs)?

- A) Purely SISD
- B) SIMD parallelism
- C) Purely MIMD
- D) Scalar parallelism

54. Why do GPUs heavily rely on hardware multithreading?

- A) Because GPUs do not have any internal memory.
- B) To avoid stalls on memory accesses while maintaining very high rates of data movement.
- C) Because GPUs only execute one thread per image.
- D) To simulate a von Neumann bottleneck.

55. What defines a Multiple-Instruction Multiple-Data (MIMD) system?

- A) A single control unit broadcasting to multiple ALUs.
- B) Multiple simultaneous instruction streams operating on multiple data streams.
- C) A system that executes one instruction at a time on one data point.
- D) A system limited exclusively to distributed memory.

56. In a UMA multicore system, what does UMA stand for?

- A) Universal Memory Array
- B) Uniform Memory Access
- C) Unified Memory Architecture
- D) Unilateral Memory Allocation

57. In a NUMA system, how is memory accessed differently compared to a UMA system?

- A) NUMA systems cannot access main memory at all.
- B) A memory location to which a core is directly connected can be accessed more quickly than one accessed through another chip.
- C) All memory locations take exactly the same time to access.
- D) Memory accesses must be handled exclusively by message passing.

58. What is the most widely available distributed-memory system today, composed of commodity systems connected by a commodity network?

- A) A vector supercomputer
- B) A cluster
- C) A symmetric multiprocessor
- D) A pure crossbar switch

59. What is a major disadvantage of using a shared bus interconnect for a large number of processors?

- A) It requires writing specialized assembly code for each processor.
- B) As the number of devices increases, contention for the bus increases, decreasing expected performance.
- C) A bus cannot transmit floating-point data.
- D) A bus physically cannot reach more than two processors.

60. Which switched interconnect uses internal switches to allow simultaneous communications between different processors and memory modules, provided they do not access the same module?

- A) Ring network
- B) Linear bus
- C) Crossbar
- D) Omega network

61. How is the "bisection width" of a network defined?

- A) The physical length of the cables connecting processors.
- B) The maximum number of bits transmitted per second across the network.
- C) The minimum number of links needed to be removed to split the set of nodes into two equal halves.
- D) Total number of links.

62. What is the bisection width of a two-dimensional toroidal mesh with p nodes (where p is a square number)?

- A) 2
- B) p
- C) $p / 2$
- D) $2 * \sqrt{p}$

63. Which direct interconnect is built inductively by connecting corresponding switches of two identical smaller interconnects, resulting in $p = 2^d$ nodes?

- A) Toroidal mesh
- B) Crossbar
- C) Hypercube
- D) Omega network

64. What is a typical characteristic of a Single Program, Multiple Data (SPMD) program?

- A) It consists of entirely different executables for each core.
- B) It uses a single executable that behaves like multiple different programs through conditional branches.
- C) It only supports task-parallelism, never data-parallelism.
- D) It cannot be run on a shared-memory system.

65. What does the term "embarrassingly parallel" describe?

- A) Programs that are impossible to parallelize.
- B) Programs that can be parallelized by simply dividing the work among the processes/threads with minimal coordination.
- C) Programs that crash when executed on more than one core.
- D) Programs that require excessive lock management and synchronization.

66. In shared-memory programming, what distinguishes dynamic threads from static threads?

- A) Dynamic threads run permanently until the system reboots.
- B) Dynamic threads are forked as work requests arrive and join when work completes, whereas static threads are forked once and run until all work is completed.
- C) Static threads consume zero system resources.
- D) Dynamic threads cannot execute SPMD programs.

67. What causes nondeterminism in parallel programs?

- A) The use of a static program counter.
- B) Because threads execute independently, the time it takes for one thread to complete statements varies, so completion order cannot be predicted.
- C) Processors executing different executables simultaneously.
- D) The lack of any local cache.

68. When multiple threads attempt to simultaneously access a shared resource and the outcome depends on which thread executes first, what is this called?

- A) A deadlock
- B) A race condition
- C) Amdahl's trap
- D) Cache coherence

69. What is a block of code called if it can only be executed by one thread at a time?

- A) A critical section
- B) A parallel section
- C) A speculative branch
- D) A datapath

70. Which synchronization mechanism is a special object with hardware support used to protect critical sections by allowing a thread to "lock" and "unlock" it?

- A) A busy-wait loop
- B) A semaphore
- C) A mutual exclusion lock (mutex)
- D) A page table

71. What is a major disadvantage of using a busy-wait loop for synchronization?

- A) It requires complex API calls to the operating system.
- B) It can be very wasteful of system resources since the core is busy checking a condition instead of doing useful work.
- C) It completely halts all processes on the computer.
- D) It cannot be used in a shared-memory environment.

72. Why might the standard C function strtok be unsafe in a multithreaded program?

- A) It crashes if the input string is empty.
- B) It uses a static local variable that persists between calls and is effectively shared among threads, causing unpredictable behavior.
- C) It cannot be compiled by modern C compilers.
- D) It explicitly locks the entire memory space during execution.

73. In a typical message-passing API like MPI, what is the default behavior of a simple Send function?

- A) It permanently caches the message in the TLB.
- B) It blocks until the matching call to Receive has started receiving the data or data is copied into storage.
- C) It broadcasts the message to every active process.
- D) It writes the message directly to a shared file on the hard drive.

74. Why is message passing sometimes referred to as "the assembly language of parallel programming"?

- A) Because it strictly requires writing code in x86 assembly language.
- B) Because it manages a huge amount of low-level detail, often requiring programmers to rewrite the vast majority of the program.
- C) Because it only runs on SIMD processors.
- D) Because it was the first language developed in the 1940s.

75. When writing parallel programs with standard C I/O, which of the following is an assumed rule for distributed-memory programs?

- A) All processes should call scanf simultaneously.
- B) Only process 0 will access stdin.
- C) Processes should write their outputs to the exact same file without locks.
- D) stdout is physically disabled on parallel machines.

76. If a serial program runs in T_{serial} time, what is the ideal best possible run-time for a parallel program using p cores?

- A) $T_{\text{parallel}} = T_{\text{serial}} / p$
- B) $T_{\text{parallel}} = T_{\text{serial}} * p$
- C) $T_{\text{parallel}} = T_{\text{serial}} + p$
- D) $T_{\text{parallel}} = T_{\text{serial}} - p$

77. If a parallel program achieves $T_{\text{parallel}} = T_{\text{serial}} / p$, what is this called?

- A) Superlinear speedup
- B) Distributed speedup
- C) Linear speedup
- D) Minimal speedup

78. What causes parallel programs to usually fall short of linear speedup?

- A) The cores deliberately run at slower clock speeds.
- B) The parallel program almost invariably introduces overhead, such as critical sections or network communication.
- C) Most algorithms are mathematically impossible to divide.
- D) The operating system limits the CPU utilization to 50%.

79. How is the efficiency (E) of a parallel program calculated?

- A) $E = p / S$
- B) $E = S / p$
- C) $E = T_{\text{serial}} * p$
- D) $E = S * p$

80. What general principle does Amdahl's Law describe regarding parallel computing?

- A) As the number of processors increases, the speed of light limits data transfer.
- B) Unless virtually all of a serial program is parallelized, the possible speedup is going to be very limited, capped by the inherently serial fraction.
- C) Adding more processors magically improves the performance of any serial program.
- D) The number of transistors on a microchip doubles every two years.

81. According to Amdahl's Law, if 10% of a program is strictly serial ($r = 0.1$), what is the maximum theoretical speedup regardless of how many cores are used?

- A) 100
- B) 50

C) 10

D) 2

82. Why shouldn't programmers be overly worried by Amdahl's Law according to Gustafson's law?

A) Because modern CPUs ignore the serial fraction.

B) For many problems, as we increase the problem size, the inherently serial fraction of the program decreases in size.

C) Because we can always rewrite the serial fraction to be 0%.

D) Because Amdahl's law was proven mathematically in 2010.

83. When reporting the final performance of a parallel program, what type of timing is typically used?

A) Total CPU time

B) "Wall clock" elapsed time

C) System time reported by clock

D) Idle time

84. What is a problem with relying exclusively on CPU time for evaluating parallel program performance?

A) It ignores idle time, giving a misleading picture of run-time if a process was waiting on messages or synchronization.

B) CPU time measures in minutes rather than milliseconds.

C) It only measures the execution time of the master thread.

D) It includes time spent typing at the keyboard.

85. When recording timings of parallel programs, why is it customary to report the minimum elapsed time from several runs?

A) Because the program mathematically changes its instructions each time.

B) It is unlikely that an outside event made the program run faster than its best possible run-time, but many events could make it run slower.

C) To exaggerate the performance of the system.

D) Because the mean time cannot be calculated for floating point numbers.

86. What is the first step in Ian Foster's methodology for parallel program design?

A) Mapping

B) Agglomeration

C) Communication

D) Partitioning

87. In Foster's methodology, what is the purpose of the Agglomeration (or aggregation) step?

A) To compile the C code into an executable.

B) To combine small tasks and communications identified earlier into larger composite tasks.

C) To explicitly assign tasks to threads.

D) To determine the bisection bandwidth of the network.

88. In Foster's methodology, what does the Mapping step involve?

A) Dividing the computation into the smallest possible tasks.

B) Assigning the composite tasks to processes/threads in a way that minimizes communication and balances the workload.

C) Converting serial loops into parallel loops automatically.

D) Determining what communication needs to be carried out among tasks.

89. When parallelizing a histogram program using an array for global bin counts on a MIMD system, why might it be bad for all threads to directly increment the shared array?

A) It requires a continuous network connection to the internet.

B) It will result in race conditions, and enforcing mutual exclusion on every update could be very expensive.

C) MIMD systems do not support integer arrays.

D) It would cause a page fault in virtual memory.

90. How can the histogram race condition be avoided efficiently on a MIMD system?

A) By completely eliminating the bin_counts array.

B) By using temporary per-process or per-thread storage (e.g., loc_bin_counts), then carrying out a global sum at the end.

C) By forcing the program to run serially on a single core.

D) By switching to a SIMD architecture exclusively.

Part2: sheet

1. Instead of building faster monolithic processors, manufacturers shifted to:

a) Larger cache sizes

b) Multicore processors

c) Analog computers

d) Virtual processors

2. A program written to run on a single processor is called a:

a) Parallel program

b) Concurrent program

c) Serial program

d) Distributed program

3. Running multiple instances of a serial program on a multicore system is often not helpful because:

a) It damages the hardware

b) Users want a single program to run faster (e.g., better graphics)

c) The operating system cannot handle it

d) It consumes too much memory

4. Attempts to automatically translate serial programs into parallel programs have:

a) highly successful

b) moderately successful

c) Had very limited success

d) abandoned completely

5. Two widely used approaches for partitioning work among cores are:

a) Input-parallelism and Output-parallelism

b) Task-parallelism and Data-parallelism

c) Static-parallelism and Dynamic-parallelism

d) Hardware-parallelism and Software-parallelism

6. Partitioning a large array among cores where each core applies similar operations on its part is an example of:

- a) Task-parallelism
- b) Data-parallelism
- c) Functional-parallelism
- d) Pipeline-parallelism

7. Dividing grading by question number (Q1 to TA A, Q2 to TA B, etc.) is an example of:

- a) Data-parallelism
- b) Task-parallelism
- c) Load balancing
- d) Synchronization

8. Coordination where one or more cores send their partial sums to another core is an example of:

- a) Load balancing
- b) Communication
- c) Caching
- d) Virtualization

9. Ensuring each core is assigned roughly the same amount of work is known as:

- a) Synchronization
- b) Load balancing
- c) Communication
- d) Partitioning

10. The mechanism to prevent cores from racing ahead before a master core has initialized data is called:

- a) Load balancing
- b) Communication
- c) Synchronization
- d) Partitioning

11. Which API is designed for programming distributed-memory MIMD systems?

- a) OpenMP
- b) Pthreads
- c) CUDA
- d) MPI

12. Which API is designed for programming shared-memory MIMD systems?

- a) MPI
- b) Pthreads
- c) CUDA
- d) Verilog

13. In a shared-memory system, cores coordinate their work by:

- a) Sending messages over a network
- b) Updating shared memory locations
- c) Using a master control unit

d) Using separate instruction streams only

14. In a distributed-memory system, cores communicate explicitly by:

- a) Reading global variables
- b) Sending messages across a network
- c) Using shared caches
- d) Sharing a system bus

15. A system where all cores carry out the same instruction on their own data is called:

- a) SISD
- b) MIMD
- c) SIMD
- d) MISD

16. A system where each core has its own control unit and can execute independent instruction streams is called:

- a) SISD
- b) SIMD
- c) MIMD
- d) MISD

17. Current generation GPUs rely heavily on which type of parallelism?

- a) MIMD
- b) SIMD
- c) SISD
- d) MISD

18. According to the text, a program in which multiple tasks can be in progress at any instant is called:

- a) Parallel
- b) Distributed
- c) Concurrent
- d) Serial

19. A program where multiple tasks cooperate closely to solve a problem is best defined as:

- a) Concurrent
- b) Parallel
- c) Distributed
- d) Multitasking

20. Programs that may cooperate but are loosely coupled and often geographically separated are called:

- a) Parallel
- b) Distributed
- c) Concurrent
- d) Integrated

21. The term 'core' in multicore processors has become synonymous with:

- a) Memory Unit

b) CPU (Central Processing Unit)

c) Cache

d) Bus

22. The separation of memory and CPU in the von Neumann architecture is often called the:

a) Memory Wall

b) Von Neumann Bottleneck

c) CPU Latency

d) Bus Error

23. What technology stores blocks of data/instructions in faster memory close to the CPU?

a) Virtual Memory

b) Caching

c) Pipelining

d) Multithreading

24. When a CPU attempts to read data and it is found in the cache, it is called a:

a) Cache Miss

b) Cache Hit

c) Page Fault

d) Stall

25. In a write-through cache:

a) Data is written only to the cache and marked dirty

b) Data is written to main memory when the line is evicted

c) Data is written to both cache and main memory simultaneously

d) Data is never written to main memory

26. A cache where each line of memory maps to a unique location is called:

a) Fully Associative

b) n-way Set Associative

c) Direct Mapped

d) Indirect Mapped

27. The scheme used to decide which cache line to evict is usually:

a) First In First Out (FIFO)

b) Least Recently Used (LRU)

c) Random

d) Most Recently Used (MRU)

28. C stores two-dimensional arrays in:

a) Column-major order

b) Row-major order

c) Diagonal order

d) Random order

29. Virtual memory allows main memory to function as a cache for:

a) The CPU registers

b) The L1 Cache

- c) Secondary storage (swap space)
- d) The Network

30. A block of data in virtual memory is called a:

- a) Cache line
- b) Sector
- c) Page
- d) Segment

31. The hardware structure that caches entries from the page table to speed up address translation is the:

- a) L2 Cache
- b) Program Counter
- c) Translation-Lookaside Buffer (TLB)
- d) Memory Management Unit

32. Instruction-level parallelism (ILP) includes which two main approaches?

- a) Caching and Virtual Memory
- b) Pipelining and Multiple Issue
- c) SIMD and MIMD
- d) Blocking and Non-blocking

33. Breaking instruction execution into stages like fetch, decode, and execute to run them simultaneously is called:

- a) Multiple Issue
- b) Pipelining
- c) Multithreading
- d) Vectorization

34. Processors that replicate functional units to execute multiple instructions simultaneously are often called:

- a) Superscalar
- b) Vector processors
- c) Scalar
- d) Pipelined

35. Simultaneous Multithreading (SMT) is a variation of:

- a) Fine-grained multithreading
- b) Coarse-grained multithreading
- c) Pipelining
- d) Virtual Memory

36. According to Flynn's Taxonomy, a traditional serial computer is:

- a) SIMD
- b) MIMD
- c) SISD
- d) MISD

37. A system where one control unit broadcasts instructions to multiple datapaths is:

- a) SISD
- b) SIMD**
- c) MIMD
- d) MISD

38. Vector processors are a type of:

- a) SISD system
- b) MIMD system
- c) SIMD system**
- d) Cluster

39. What is 'scatter/gather' in the context of vector processors?

- a) Writing/Reading elements located at irregular intervals**
- b) Sending data to all processors
- c) Collecting results from all processors
- d) Distributing tasks randomly

40. Graphics Processing Units (GPUs) primarily use which type of parallelism?

- a) SISD
- b) SIMD**
- c) Pure MIMD
- d) Sequential

41. Shared-memory systems where access time to all memory locations is the same are called:

- a) NUMA
- b) UMA**
- c) Clusters
- d) Vector Systems

42. In distributed-memory systems, processors communicate by:

- a) Updating shared variables
- b) Sending messages**
- c) Using a common bus
- d) Using cache coherence protocols

43. When multiple threads attempt to update a shared resource simultaneously, it is called a:

- a) Deadlock
- b) Race condition**
- c) Cache miss
- d) Segmentation fault

44. A block of code that can only be executed by one thread at a time is called a:

- a) Parallel region
- b) Serial region
- c) Critical section**
- d) Loop

45. Ideally, if a parallel program runs on p cores, the speedup should be:

- a) p (Linear Speedup)**

- b) $2p$
- c) p^2
- d) 1

46. The ratio of serial run-time to parallel run-time is called:

- a) Efficiency
- b) Speedup
- c) Scalability
- d) Throughput

47. Efficiency is defined as:

- a) Speedup / p
- b) p / Speedup
- c) Speedup * p
- d) $T_{\text{parallel}} * T_{\text{serial}}$

48. Amdahl's Law states that the maximum speedup is limited by:

- a) The number of processors
- b) The size of the memory
- c) The fraction of the program that is inherently serial
- d) The speed of the interconnect

49. If 10% of a program is serial, the maximum possible speedup according to Amdahl's Law is:

- a) 5
- b) 10
- c) 20
- d) 100

50. In Foster's methodology, dividing the computation and data into small tasks is called:

- a) Partitioning
- b) Mapping
- c) Agglomeration
- d) Communication

51. In Foster's methodology, assigning tasks to processes/threads is called:

- a) Partitioning
- b) Agglomeration
- c) Mapping
- d) Communication

52. Typically, in a ring interconnect with p processors, the number of links is:

- a) p
- b) $2p$
- c) p^2
- d) $p/2$

53. Busy-waiting is a mechanism where:

- a) A thread sleeps until a condition is met
- b) A thread repeatedly tests a condition in a loop
- c) A thread is blocked by the OS
- d) A thread terminates immediately

54. A function is 'thread-safe' if:

- a) It can be called from a serial program
- b) It uses static variables
- c) It can be safely called by multiple threads simultaneously
- d) It only uses local variables

55. In a distributed-memory program, the 'assembly language of parallel programming' refers to:

- a) OpenMP
- b) Java
- c) Message-Passing (MPI)
- d) Python

56. In the static thread paradigm:

- a) Threads are forked and joined for every task
- b) Threads are created once and run until the work is completed
- c) Threads are created by the hardware
- d) Threads are never used

57. An operation that writes to memory is atomic if:

- a) It happens instantaneously
- b) It cannot be interrupted
- c) No other thread sees the intermediate state
- d) It is very fast

58. To ensure mutual exclusion in a critical section, a commonly used mechanism is:

- a) A loop
- b) A Mutex (Lock)
- c) A global variable
- d) A pointer

59. Semaphores are similar to mutexes but:

- a) Are slower
- b) Are only used in serial programs
- c) Can handle different details of thread synchronization
- d) Are not supported by hardware

60. In a shared memory system, the interconnect connecting all processors directly to main memory results in:

- a) NUMA
- b) UMA
- c) Distributed Memory
- d) A Cluster

61. Write-back caches use a 'dirty' bit to indicate:

- a) The cache line is empty
- b) The cache line has been updated and differs from main memory
- c) The cache line is corrupted
- d) The cache line is read-only

62. Hardware multithreading allows a system to:

- a) Run faster clock speeds
- b) Switch threads when the current task stalls (e.g., waiting for memory)
- c) Execute multiple instructions from the same thread simultaneously
- d) Use more power

63. A 'Page Fault' occurs when:

- a) The CPU crashes
- b) A requested page is not in memory and must be fetched from disk
- c) The TLB misses
- d) A cache line is evicted

64. A sequence of basic steps for solving a problem using a serial computer is a:

- a) Parallel algorithm
- b) Sequential algorithm
- c) Decomposition
- d) Mapping

65. The process of dividing a computation into smaller parts for parallel execution is called:

- a) Mapping
- b) Agglomeration
- c) Decomposition
- d) Synchronization

66. Programmer-defined units of computation are called:

- a) Threads
- b) Processes
- c) Tasks
- d) Jobs

67. A directed acyclic graph where nodes represent tasks and edges indicate dependencies is a:

- a) Task-interaction graph
- b) Task-dependency graph
- c) Process graph
- d) Communication graph

68. A decomposition into a large number of small tasks is called:

- a) Coarse-grained
- b) Fine-grained
- c) Static
- d) Dynamic

69. The maximum number of tasks that can be executed simultaneously is the:

- a) Average degree of concurrency
- b) Maximum degree of concurrency
- c) Critical path length
- d) Granularity

70. The mechanism by which tasks are assigned to processes is called:

- a) Decomposition
- b) Partitioning
- c) Mapping
- d) Scheduling

71. Decomposition based on the divide-and-conquer strategy is called:

- a) Data decomposition
- b) Recursive decomposition
- c) Exploratory decomposition
- d) Speculative decomposition

72. Quicksort is typically parallelized using:

- a) Data decomposition
- b) Recursive decomposition
- c) Speculative decomposition
- d) Exploratory decomposition

73. Partitioning the data to induce a decomposition of computations is called:

- a) Recursive decomposition
- b) Data decomposition
- c) Functional decomposition
- d) Task decomposition

74. The 'Owner-Computes Rule' generally means:

- a) The process that owns the data performs computations involving that data
- b) The master process performs all computations
- c) The user computes the data manually
- d) The output data determines the input data

75. Which decomposition is used for searching a space for solutions (e.g., 15-puzzle)?

- a) Data decomposition
- b) Recursive decomposition
- c) Exploratory decomposition
- d) Speculative decomposition

76. A decomposition where tasks execute potential future branches before the condition is evaluated is:

- a) Exploratory decomposition
- b) Speculative decomposition
- c) Recursive decomposition
- d) Data decomposition

77. Discrete event simulation is an example application for:

- a) Data decomposition
- b) Recursive decomposition
- c) Speculative decomposition
- d) Static mapping

78. If task sizes are known a priori, which mapping technique is usually suitable?

- a) Dynamic mapping
- b) Static mapping
- c) Random mapping
- d) Speculativ mapping

79. If task sizes are unknown or dynamic, which mapping is more effective?

- a) Static mapping
- b) Dynamic mapping
- c) Block mapping
- d) Cyclic mapping

80. Which distribution is used to alleviate load-imbalance in LU factorization?

- a) Block distribution
- b) Block-cyclic distribution
- c) Single element distribution
- d) Random distribution

81. A cyclic distribution is an extreme case of:

- a) Block distribution
- b) Block-cyclic distribution
- c) Random distribution
- d) Tree distribution

82. Randomized block distribution is useful when:

- a) The distribution of work has special patterns (e.g., sparse matrices)
- b) The matrix is dense
- c) The tasks are uniform
- d) Communication is zero

83. Mapping based on partitioning a task-dependency graph is called:

- a) Data partitioning
- b) Task partitioning
- c) Random partitioning
- d) Cyclic partitioning

84. Static task generation means:

- a) Tasks are generated during execution
- b) All tasks are known before algorithm execution
- c) Tasks are random
- d) Tasks are always uniform

85. Instances where the set of communicating tasks is known prior to execution are called:

- a) Dynamic interactions

- b) Static interactions
- c) Random interactions
- d) Irregular interactions

86. Image dithering is an example of:

- a) Irregular interaction
- b) Regular interaction
- c) Dynamic interaction
- d) One-way interaction

87. In distributed dynamic load balancing, processes:

- a) Use a central queue
- b) Exchange tasks at run time
- c) Do not communicate
- d) Use static mapping

88. Receiver-initiated load balancing works well when:

- a) The sender has too much work
- b) The receiver runs out of work
- c) The network is slow
- d) Tasks are uniform

89. Matrix multiplication $C = A \times B$ can be decomposed by partitioning:

- a) Only Matrix A
- b) Only Matrix B
- c) The output matrix C
- d) The compiler

90. Block distributions are particularly suitable when there is:

- a) Locality of interaction
- b) Random interaction
- c) No interaction
- d) Dynamic task generation

91. In a 2D block distribution of a matrix, processes are arranged in a:

- a) Linear array
- b) Grid
- c) Tree
- d) Ring

92. Which distribution usually incurs a lower volume of inter-task interaction?

- a) One-dimensional distribution
- b) Higher dimensional distribution
- c) Random distribution
- d) Cyclic distribution

93. Hierarchical mappings are useful when:

- a) Tasks are uniform
- b) The task-dependency graph has limited concurrency at the top

- c) No communication is needed
- d) The system is SISD

94. The primary reason for using dynamic mapping is:

- a) Reducing communication
- b) Balancing workload
- c) Simplifying code
- d) Using static arrays

95. If task sizes vary significantly, tasks are said to be:

- a) Uniform
- b) Non-uniform
- c) Static
- d) Dynamic

96. When implementing a parallel algorithm with dynamic interactions in message-passing, tasks must often use:

- a) Polling
- b) Static arrays
- c) Read-only memory
- d) Hard drives

136. MPI stands for:

- a) Multiple Processor Interface
- b) Message-Passing Interface
- c) Memory-Passing Interface
- d) Main Program Interface

97. In MPI, a collection of processes that can send messages to each other is called a:

- a) Group
- b) Communicator
- c) Cluster
- d) World

98. The default communicator that consists of all started processes is:

- a) MPI_ALL
- b) MPI_COMM_WORLD
- c) MPI_WORLD
- d) MPI_DEFAULT

99. The function 'MPI_Init':

- a) Sends a message
- b) Initializes the MPI system
- c) Finalizes the MPI system
- d) Returns the process rank

100. The function 'MPI_Finalize':

- a) Stops the program immediately
- b) Cleans up resources and ends MPI

- c) Sends the last message
- d) Prints the output

101. Which function returns the number of processes in a communicator?

- a) MPI_Comm_rank
- b) MPI_Comm_size
- c) MPI_Size
- d) MPI_Count

102. Which function returns the calling process's rank?

- a) MPI_Comm_rank
- b) MPI_Comm_size
- c) MPI_Rank
- d) MPI_Get_rank

103. Most MPI programs are written using which model?

- a) MPMD
- b) SPMD (Single Program, Multiple Data)
- c) SIMD
- d) SISD

104. The 'MPI_Send' function's 'dest' argument specifies:

- a) The memory address
- b) The rank of the receiving process
- c) The size of the message
- d) The tag

105. The 'tag' argument in 'MPI_Send' is used to:

- a) Identify the process
- b) Distinguish messages that might otherwise look identical
- c) Specify the size
- d) Specify the data type

106. Can a message sent with one communicator be received using a different communicator?

- a) Yes
- b) No
- c) Only if tags match
- d) Only if ranks match

107. 'MPI_Recv' returns when:

- a) The function is called
- b) A matching message has been received
- c) The buffer is empty
- d) The sender calls MPI_Finalize

108. Can a sender use a wildcard for the destination?

- a) Yes
- b) No
- c) Only for 'MPI_Bcast'

d) Only for 'MPI_Reduce'

109. If 'MPI_Send' buffers the message, the call returns:

- a) When the receiver gets the message
- b) Immediately after the message is placed in the buffer
- c) Never
- d) After MPI_Finalize

110. If 'MPI_Send' blocks, it waits until:

- a) The message is buffered or transmission begins
- b) The receiver processes the data
- c) The program ends
- d) The tag is checked

111. Messages in MPI are non-overtaking, meaning:

- 3) Messages from different processes arrive in order
- b) Messages from the *same* sender arrive in the order sent
- c) Messages are always faster than calculation
- d) Messages never arrive

112. 'MPI_Reduce' is used to:

- a) Send a message to one process
- b) Combine results from all processes into a single result (e.g., sum)
- c) Divide the number of processes
- d) Stop the program

113. Which 'MPI_Op' is used for summation?

- a) MPI_MAX
- b) MPI_SUM
- c) MPI_PROD
- d) MPI_ADD

114. In 'MPI_Reduce', the 'root process' argument specifies:

- a) The process that sends the data
- b) The process that receives the final result
- c) The number of processes
- d) The tag

115. Is it legal to use the same buffer for input and output in 'MPI_Reduce' (aliasing)?

- a) Yes
- b) No
- c) Only for integers
- d) Only on process 0

116. 'MPI_Allreduce' differs from 'MPI_Reduce' in that:

- a) It uses a different operator
- b) The result is stored on *all* processes
- c) It is faster
- d) It uses point-to-point communication

117. MPI **Bcast** is used to:

- a) Send different data to all processes
- b) Send the ***same*** data from one process to all others
- c) Collect data from all processes
- d) Reduce data

118. In **'MPI_Bcast'**, the **'data_p'** argument is:

- a) Always input
- b) Always output
- c) **Input on the source, output on others**
- d) Ignored

119. **'MPI_Scatter'** is used to:

- a) Broadcast the same data to everyone
- b) **Divide data into pieces and send different pieces to different processes**
- c) Collect data onto one process
- d) Randomize data

120. **'MPI_Gather'** is the inverse of:

- a) **'MPI_Bcast'**
- b) **'MPI_Scatter'**
- c) **'MPI_Reduce'**
- d) **'MPI_Send'**

121. **'MPI_Allgather'** does what?

- a) Gathers data to one process
- b) **Concatenates data from all processes and distributes the complete result to all processes**
- c) Broadcasts a single value
- d) Adds values together

122. **'MPI_Wtime'** returns:

- a) CPU time
- b) **Wall clock time (elapsed time)**
- c) The current date
- d) The processor speed

123. **'MPI_Barrier'** ensures that:

- a) **No process returns until all have started the call**
- b) Processes stop forever
- c) Data is saved
- d) Errors are checked

124. A **communication pattern** where processes exchange partial results **(like a reverse tree)** is called a:

- a) Ring
- b) **Butterfly**
- c) Star
- d) Mesh

125. Which MPI constant acts as a **null process** rank for boundary conditions (like in odd-even sort)?

- a) 'MPI_NULL'
- b) 'MPI_PROC_NULL'
- c) 'MPI_NONE'
- d) 'MPI_EMPTY'

126. A derived **datatype** consists of a sequence of basic types and:

- a) Their order is ignored
- b) Their **displacements (relative locations)**
- c) Their names
- d) Their sizes only

127. Which function **gets** the **address** of a memory location for building datatypes?

- a) 'MPI_Address'
- b) 'MPI_Get_address'
- c) 'MPI_Locate'
- d) 'MPI_Pointer'

128. When **running MPI** programs, which command is typically used to start them?

- a) 'start_mpi'
- b) 'mpiexec or mpirun'
- c) 'run'
- d) 'execute'

129. If 'MPI_Recv' uses 'MPI_STATUS_IGNORE', it means:

- a) The **receiver** doesn't care about the **status** (source, tag, error)
- b) The receiver will ignore the message
- c) The status is stored
- d) The status is invalid

130. In the matrix-**vector** multiplication example, 'MPI_Allgather' is used to:

- a) Distribute the matrix A
- b) **Collect the entire vector x onto every process**
- c) Calculate the dot product
- d) Print the result

131. Which function **cleans up** a committed derived datatype?

- a) 'MPI_Type_delete'
- b) 'MPI_Type_free'
- c) 'MPI_Type_remove'
- d) 'MPI_Type_clean'

132. Parallel efficiency is typically:

- a) Greater than 1
- b) **Less than or equal to 1**
- c) Exactly 0
- d) Infinite

133. Linear speedup implies an efficiency of:

- a) 0.5
- b) 1.0
- c) 0.0
- d) p

134. Ideally, T_{parallel} equals:

- a) T_{serial}
- b) T_{serial} / p
- c) $T_{\text{serial}} * p$
- d) T_{overhead}

135. The 'operator' argument in 'MPI_Reduce' (e.g., MPI_SUM) must be:

- a) Commutative
- b) Associative
- c) Associative (and theoretically commutative for some implementations)
- d) Recursive

136. For input (scanf), most MPI implementations allow stdin access by:

- a) All processes
- b) Only process 0
- c) Only the last process
- d) No processes

137. For output (printf), most MPI implementations allow stdout access by:

- a) Only process 0
- b) All processes (behavior is nondeterministic)
- c) Only process 1
- d) No processes

138. To compile an MPI program in C, the command is typically:

- a) 'gcc'
- b) 'mpicc'
- c) 'javac'
- d) 'cc'

139. MPI identifiers always start with:

- a) 'M_'
- b) 'MPI_'
- c) 'Parallel_'
- d) 'MP_'

140. Which header file must be included in C MPI programs?

- a) <stdio.h>
- b) <mpi.h>
- c) <parallel.h>
- d) <msg.h>

141. In the parallel vector sum example, dividing n components into blocks of $n/\text{comm_sz}$ is a:

- a) Cyclic partition
- b) Block partition
- c) Random partition
- d) Tree partition

142. If process 0 reads a vector and sends only needed components to other processes, it avoids:

- a) Computation
- b) Wasteful memory allocation on other processes
- c) Using MPI
- d) Parallelism

143. 'recv_count' in 'MPI_Gather' specifies:

- a) The total data received
- b) The number of items received from *each* process
- c) The rank of the receiver
- d) The size of the communicator

144. In matrix-vector multiplication with row-block partition, the output vector y is partitioned by:

- a) Blocks
- b) Cyclic
- c) It is not partitioned
- d) Randomly

Part 3: True / False Questions (40 Questions)

1. Statement: Between 1986 and 2003, microprocessor performance increased, on average, more than 50% per year.

2. Statement: The shift to multicore processors was largely driven by the inability to effectively dissipate the heat generated by ever-faster, denser, single monolithic processors.

3. Statement: Researchers have had great success writing programs that automatically and perfectly convert serial C/C++ programs into efficient parallel programs.

4. Statement: Task-parallelism partitions the data used in solving the problem among the cores, whereas data-parallelism partitions the various tasks.

5. Statement: OpenMP is an API designed exclusively for programming distributed-memory systems.

6. Statement: In a shared-memory system, the cores coordinate their work by implicitly modifying shared memory locations.

7. Statement: In concurrent computing, multiple tasks can be in progress at any instant, meaning even a single-core multitasking OS can be considered concurrent.

8. Statement: The "von Neumann bottleneck" refers to the limitation caused by the interconnect between the CPU and main memory determining the rate at which data and instructions are accessed.

9. Statement: A process consists of the executable code, a call stack, a heap, resources, security information, and process state information.

10. Statement: Threads are "lighter weight" than processes because two threads belonging to one process can share most of the process's resources.

11. Statement: The principle of locality relies on the idea that programs tend to use data and instructions that are physically close to recently used data and instructions.

12. Statement: A CPU cache typically handles data one single byte at a time rather than in larger blocks.

13. Statement: In a write-through cache, modified data is not written to main memory until the cache line is evicted.

14. Statement: Virtual memory functions as a cache for secondary storage by keeping only the active parts of running programs in main memory.

15. Statement: A Translation-Lookaside Buffer (TLB) is used to cache entries from the page table to speed up virtual memory translation.

16. Statement: Pipelining improves processor performance by allowing multiple instructions to simultaneously execute across replicated functional units, whereas multiple issue arranges functional units in sequential stages.

17. Statement: Speculation is a technique where the compiler or processor makes a guess about an instruction's outcome and executes it based on that guess.

18. Statement: Coarse-grained multithreading attempts to avoid wasting machine time by switching threads after each individual instruction.

19. Statement: In Flynn's taxonomy, a classical von Neumann machine is considered a Single Instruction Single-Data (SISD) system.

20. Statement: In a SIMD system, different datapaths can simultaneously execute completely different instructions.

21. Statement: Vector processors typically include vectorized functional units and vector registers, allowing them to operate on arrays of data simultaneously.

22. Statement: Modern Graphics Processing Units (GPUs) rely heavily on hardware multithreading to avoid stalls on memory accesses.

23. Statement: In a Uniform Memory Access (UMA) system, the time required to access all memory locations is exactly the same for all cores.

24. Statement: A cluster is a distributed-memory system typically composed of a collection of commodity systems connected by a commodity interconnection network.

25. Statement: A crossbar is an inexpensive interconnect ideally suited for connecting thousands of processors without conflict.

26. Statement: The bisection width of a network is the minimum number of links that must be removed to split the set of nodes into two equal halves.

27. Statement: An omega network is an example of an indirect interconnect that uses a switching network to route data.

28. Statement: Single Program, Multiple Data (SPMD) programs require a completely unique executable file for every processor running in the system.

29. Statement: Programs that can be parallelized simply by dividing work among processes with little to no communication are sometimes called "embarrassingly parallel."

30. Statement: In a dynamic thread paradigm, a master thread forks worker threads as new requests arrive, and these threads terminate and join the master when the work is complete.

31. Statement: Nondeterminism in parallel computing means that a given input can result in different outputs or behaviors from run to run due to unpredictable thread execution rates.

32. Statement: A race condition occurs when multiple threads attempt to simultaneously access a shared resource, and the result depends on which thread executes its instructions first.

33. Statement: A mutual exclusion lock (mutex) forces multiple threads to execute the critical section simultaneously.

34. Statement: A busy-wait loop consumes system resources because the core repeatedly checks a condition instead of idling or doing useful work.

35. Statement: Using standard C I/O functions like printf from multiple processes or threads guarantees that the output will appear ordered and unbroken.

36. Statement: In a parallel program, linear speedup is achieved when $T_{\text{parallel}} = T_{\text{serial}} / p$, meaning the program runs exactly p times faster on p cores.

37. Statement: Parallel overhead, such as the time spent transmitting data across a network or waiting on locks, usually decreases as the number of processors increases.

38. Statement: Amdahl's Law states that the maximum possible speedup of a parallel program is limited by the fraction of the program that cannot be parallelized.

39. Statement: When taking timings of parallel programs to evaluate performance, CPU time is generally preferred over wall clock time because it accounts for idle processes waiting on locks.

40. Statement: The four steps of Foster's methodology for parallel program design are Partitioning, Communication, Agglomeration, and Mapping.

وصلنا لآخر صفحة في الملف... وآخر مادة في الكلية
بين محاضرات وامتحانات ومشاريع وضحك وتوتر اتعملت ذكريات عمرها ما هتتعوض
يمكن بعد الامتحان ده كل واحد هيمشي في طريق ومش هنجتمع كل يوم زي الأول لكن أكيد هيفضل جوانا
جزء متعلق بالأيام دي وبالناس اللي شاركونا الرحلة
أتمنى من قلبي النجاح والتوفيق لكل واحد فيكم وإن ربنا يفتحلكم أبواب الخير والسعادة من أوسع أبوابها
هتوحشوني أكثر مما تتخيلوا وهتفضلوا أجمل جزء في سنين الكلية كلها ربنا يجمعنا دايماً على خير ويكتب
لكل واحد فينا نهاية تفرحه وبداية تليق بتعبه
وما تنسونيش من صالح دعواتكم □

Hoda Hesham Hassouna